

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE

Department of Computer Science Engineering

Course Code: 23CS002PC304

Course Title: AI Assisted Coding

Year/Sem: III / II

Assignment Number: 18.1 / 24

Name: O.Narasimha Raju

HT No: 2303A53034

Batch: 46

Lab Objectives

- Understand Minimum Spanning Tree (MST)
- Implement Prim's and Kruskal's algorithms
- Solve real-world network problems
- Compare algorithm performance

Task 1: Prim's Algorithm

Python Code

```
import heapq

def prim(graph, start):
    visited = set()
    min_heap = [(0, start)]
    total_cost = 0

    while min_heap:
        cost, node = heapq.heappop(min_heap)
```

```

        if node not in visited:
            visited.add(node)
            total_cost += cost

        for neighbor, weight in graph[node]:
            if neighbor not in visited:
                heapq.heappush(min_heap, (weight, neighbor))

    return total_cost

graph = {
    0: [(1,2),(3,6)],
    1: [(0,2),(2,3),(3,8),(4,5)],
    2: [(1,3),(4,7)],
    3: [(0,6),(1,8),(4,9)],
    4: [(1,5),(2,7),(3,9)]
}

print("MST Cost =", prim(graph, 0))

```

Task 2: Kruskal's Algorithm

Python Code

```

def find(parent, i):
    if parent[i] != i:
        parent[i] = find(parent, parent[i])
    return parent[i]

def union(parent, rank, x, y):
    rootX = find(parent, x)
    rootY = find(parent, y)

    if rank[rootX] < rank[rootY]:
        parent[rootX] = rootY
    else:
        parent[rootY] = rootX
        if rank[rootX] == rank[rootY]:
            rank[rootX] += 1

```

```
def kruskal(edges, n):
    edges.sort(key=lambda x: x[2])
    parent = list(range(n))
    rank = [0] * n
    cost = 0

    for u, v, w in edges:
        if find(parent, u) != find(parent, v):
            union(parent, rank, u, v)
            cost += w

    return cost

edges = [
    (0,1,10),(0,2,6),(0,3,5),
    (1,3,15),(2,3,4)
]

print("MST Cost =", kruskal(edges, 4))
```

Task 3: Comparison

- Prim's: Suitable for dense graphs
- Kruskal's: Suitable for sparse graphs
- Both ensure minimum total cost

Task 4: Real-World Application

- Network design
- Road construction
- Power grid optimization

Conclusion

MST algorithms help minimize cost in network design problems.

AI tools assist in implementing efficient graph algorithms quickly.